
Manutenzione preventiva di un esacottero DJI F550

Messa in funzione, configurazione PX4 e campagna di acquisizione dati

Team

Angelo • Matteo • Federico

Piattaforma: DJI F550 · Pixhawk 6X · firmware PX4 · QGroundControl

16 giugno 2026

Indice

1	Introduzione	1
1.1	Obiettivo del progetto	1
1.2	Contesto del corso e metodo di lavoro	1
1.3	Sintesi del sistema	2
1.4	Struttura della relazione	2
2	Il sistema	3
2.1	Architettura generale	3
2.2	Sottosistemi	4
2.2.1	Struttura e propulsione	4
2.2.2	Avionica e navigazione	4
2.2.3	Alimentazione	4
2.2.4	Comunicazione e controllo	4
2.3	Sintesi della Bill of Materials	5
3	Configurazione: QGroundControl e firmware PX4	7
3.1	Calibrazioni e setup iniziale	7
3.2	Migrazione del GPS da CAN a UART	8
3.3	Power module e calibrazione della batteria	9
3.4	Telemetria ESC su TELEM2	9
3.5	Modalità di volo e failsafe	10
3.6	Limitazioni e problematiche riscontrate	10
4	Acquisizione dati	12
4.1	Il formato ULog di PX4	12
4.2	Architettura uORB e pipeline di logging	12
4.3	Profilo di logging SDLOG_PROFILE = 849	13
4.4	Frequenze di acquisizione	13
4.4.1	FIFO inerziali ad alta frequenza	13
4.4.2	Grandezze chiave per la manutenzione preventiva	14
4.5	Qualità del logging e dropout	14

5	Campagna di volo	15
5.1	Metodologia	15
5.2	Numerazione dei motori e tipi di traiettoria	16
5.3	Quadro delle sessioni e dei Set	17
5.4	Condizioni operative e voli scartati	17
6	Casi diagnostici	19
6.1	Sostituzione del flight controller	19
6.2	Messa a punto del collegamento GPS	19
6.2.1	Mancato aggancio dopo la migrazione a UART	20
6.2.2	Dropout del GPS in volo: diagnosi e soluzione	20
6.3	RTK non utilizzabile: voli con il solo GPS	21
6.3.1	Evidenza A/B: la preconditione è il modo RTK	21
6.4	Quasi-incidente da perdita GPS e failsafe a due livelli	22
6.5	Analisi forense dell'incidente in volo	23
7	Osservazioni sui dati	26
7.1	Metodo	26
7.2	Vibrazione e squilibrio: baseline / 5 % / 10 %	26
7.3	Progressione monotona su M1	27
7.4	Metriche raccomandate	27
8	Foxglove e interfaccia di visualizzazione	29
8.1	Motivazione	29
8.2	Pipeline <code>.ulg</code> → <code>.mcap</code>	29
8.3	Modello 3D e scena	30
8.4	Layout dei pannelli	30
9	Conclusioni	31
9.1	Risultati principali	31
9.2	Limiti e sviluppi futuri	31
A	Appendici	33
A.1	Mappatura dei log per sessione	33
A.2	Parametri PX4 rilevanti	34
A.3	Riferimenti al repository	34

Capitolo 1

Introduzione

1.1 Obiettivo del progetto

Questa relazione documenta le attività di **manutenzione preventiva** svolte su un esacottero **DJI F550** equipaggiato con **Pixhawk 6X** e firmware **PX4**, nell’ambito dell’omonimo corso (A.A. 2025/2026). Il progetto non si limita alla manutenzione in senso stretto: esso ripercorre l’intero ciclo di *messa in funzione* del velivolo, dalla configurazione del flight controller fino all’esecuzione di una campagna di voli mirata all’*acquisizione di dati* utili a caratterizzare lo stato di salute del sistema.

Il filo conduttore del lavoro è riassumibile in tre fasi:

1. **mettere in funzione** il drone, partendo da una piattaforma hardware da assemblare e diagnosticare;
2. **configurarlo** tramite QGroundControl e i parametri PX4, fino a ottenere un velivolo volabile e strumentato per il logging;
3. **eseguire i test acquisendo dati**, conducendo una campagna di voli che comprende baseline a pale sane e prove con pale volutamente danneggiate, su traiettorie di hover e quadrato.

La prospettiva della manutenzione preventiva, tema del corso, attraversa tutte e tre le fasi: l’attenzione è rivolta a *strumentare* il velivolo — cioè a predisporre l’acquisizione delle grandezze che permettono di riconoscere per tempo una degradazione — più che a riparare a guasto avvenuto. In quest’ottica ogni anomalia incontrata (dropout GPS, blocco del flight controller, incidente in volo) viene documentata insieme alla diagnosi e all’azione correttiva adottata.

1.2 Contesto del corso e metodo di lavoro

Il lavoro è stato condotto da un team di tre persone (Angelo, Matteo, Federico) nell’arco di più sessioni, da marzo a giugno 2026. Tutto il materiale è stato tracciato in un repository versionato che raccoglie la documentazione tecnica (*Bill of Materials*, datasheet), i piani di manutenzione, i log di volo in formato ULog di PX4 e gli strumenti di analisi (script di plotting

e una pipeline di visualizzazione 3D in Foxglove). Questa relazione è l’estratto ragionato di quel materiale.

L’impostazione scelta è **tematico-cronologica**: i capitoli sono organizzati per tema, e all’interno di ciascun tema si segue l’ordine temporale degli eventi. Questa scelta evita di frammentare lo stesso argomento su più giornate — il sottosistema GPS, ad esempio, è stato toccato in cinque sessioni diverse, ma viene trattato in modo unitario nel Capitolo 6.

1.3 Sintesi del sistema

Il velivolo è un esacottero a sei rotori su frame DJI F550, con distribuzione di potenza integrata nella piastra inferiore. Il cervello di bordo è un flight controller Pixhawk 6X, dotato di tre IMU ridondanti gestite in *sensor voting* dal firmware PX4. La navigazione si appoggia a un modulo GNSS montato su mast (per allontanarlo dalle interferenze elettromagnetiche dei motori) con supporto a correzione RTK. La configurazione, il monitoraggio e l’analisi delle missioni avvengono tramite la ground station QGroundControl, collegata via telemetria a 433 MHz. Il Capitolo 2 dettaglia l’architettura e ne riepiloga la *Bill of Materials*.

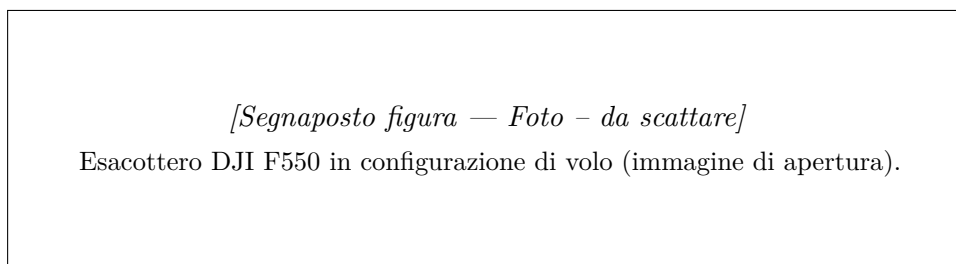


Figura 1.1: Esacottero DJI F550 in configurazione di volo (immagine di apertura).

1.4 Struttura della relazione

La relazione è organizzata come segue. Il Capitolo 2 descrive l’architettura hardware. Il Capitolo 3 ripercorre la configurazione via QGroundControl e PX4, con le problematiche incontrate. Il Capitolo 4 — cardine del progetto — spiega *cosa* viene loggato e *a che frequenza*. Il Capitolo 5 presenta la campagna di volo e tutti i tipi di prova eseguiti. Il Capitolo 6 raccoglie i casi diagnostici più significativi. Il Capitolo 7 propone alcune osservazioni sui dati acquisiti. Il Capitolo 8 presenta, come contributo extra, lo strumento di visualizzazione realizzato. Le conclusioni e le appendici chiudono il documento.

Capitolo 2

Il sistema

Questo capitolo descrive l'architettura hardware dell'esacottero e ne riepiloga la composizione tramite la *Bill of Materials* (BOM). L'obiettivo è fornire il quadro fisico del velivolo su cui poggiano tutti i capitoli successivi: la configurazione del firmware, l'acquisizione dati e la diagnosi dei guasti si riferiscono ai sottosistemi qui introdotti.

2.1 Architettura generale

Il velivolo è un **esacottero**, ossia un multirottore a sei rotori disposti simmetricamente sui bracci del frame. Rispetto a un quadricottero, la configurazione a sei motori offre maggiore capacità di carico e una ridondanza intrinseca: la perdita parziale di spinta su un rotore può essere in parte compensata dagli altri. Questa caratteristica è rilevante per il progetto, perché la campagna di volo (Capitolo 5) studia proprio l'effetto di una pala danneggiata sul comportamento del velivolo.

I sottosistemi principali sono cinque:

Struttura il frame F550 con carrello e mast GPS;

Propulsione sei gruppi motore-ESC-elica;

Avionica il flight controller Pixhawk 6X con IMU, magnetometro e GNSS;

Alimentazione batteria LiPo, distribuzione di potenza e regolatori;

Comunicazione radiocomando RC e telemetria verso la ground station.

[Segnaposto figura — Schema – da generare]

Schema a blocchi dell'architettura: Pixhawk 6X come nodo centrale collegato a GPS (UART), sei ESC con telemetria DShot, ricevitore RC, telemetria 433 MHz e power module.

Figura 2.1: Schema a blocchi dell'architettura: Pixhawk 6X come nodo centrale collegato a GPS (UART), sei ESC con telemetria DShot, ricevitore RC, telemetria 433 MHz e power module.

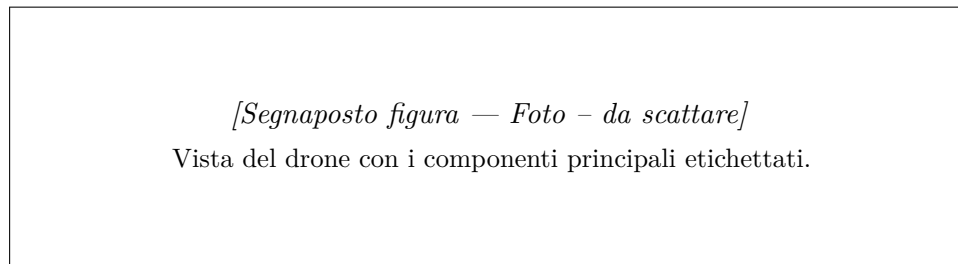


Figura 2.2: Vista del drone con i componenti principali etichettati.

2.2 Sottosistemi

2.2.1 Struttura e propulsione

Il frame DJI F550 è un telaio esagonale che integra la *Power Distribution Board* (PDB) nella piastra inferiore, semplificando il cablaggio di potenza verso i sei ESC. Il carrello è costituito da piedi flessibili ad arco che ammortizzano l'atterraggio. La propulsione è affidata a sei motori brushless **AIR2213 KV920** (tre in rotazione oraria e tre antioraria, necessari per l'equilibrio di coppia del multirobot), pilotati da sei ESC **Tekko32 F4** con telemetria DShot. Quest'ultima è particolarmente utile ai fini del progetto, perché permette di registrare regime, corrente e temperatura di ciascun motore (Capitolo 4).

2.2.2 Avionica e navigazione

Il flight controller è un **Pixhawk 6X**, basato su microcontrollore STM32H7 e progettato per il firmware PX4. Integra **tre IMU ridondanti** (accelerometro + giroscopio) gestite in *sensor voting*: il firmware confronta le letture e seleziona/scarta i sensori in caso di discrepanza, aumentando l'affidabilità della stima d'assetto. La navigazione si appoggia a un modulo **GNSS CubePilot Here+** (con magnetometro integrato) montato su un mast di circa 15 cm per allontanarlo dalle interferenze elettromagnetiche generate dai cavi di potenza e dai motori. Il sistema supporta la correzione **RTK** tramite una stazione base, per una precisione di posizionamento centimetrica.

2.2.3 Alimentazione

L'energia è fornita da una batteria **LiPo 4S da 5600 mAh** (16,8 V a piena carica, connettore XT60). La PDB distribuisce la potenza ai sei ESC; un regolatore step-down (BEC) ricava i 5 V per l'avionica e le periferiche. Il monitoraggio di tensione e corrente avviene tramite un power module collegato al Pixhawk, la cui corretta configurazione è stata oggetto di un intervento specifico (Capitolo 3).

2.2.4 Comunicazione e controllo

Il controllo manuale avviene tramite un radiocomando FrSky con ricevitore **X8R** a bordo. Il collegamento dati con la ground station **QGroundControl** è realizzato da una coppia di

moduli telemetria **Holybro a 433 MHz**, che trasmette telemetria in tempo reale e consente la modifica dei parametri e il monitoraggio dei failsafe.

2.3 Sintesi della Bill of Materials

La Tabella 2.1 riepiloga i componenti principali con i codici identificativi adottati nel repository (S struttura, P propulsione, A avionica, E alimentazione, C comunicazione, SW software). Il dettaglio completo, con note e voci ancora in via di definizione, è mantenuto nel file `docs/BOM.md`.

Tabella 2.1: Estratto della Bill of Materials dell'esacottero F550.

ID	Componente	Modello	Q.tà
S-01	Frame	DJI F550 (PDB integrata)	1
S-02	Carrello	Piedi flessibili ad arco	1 set
S-03	Mast GPS	Albero ~15 cm anti-EMI	1
P-01	Motore brushless	AIR2213 / KV920	6
P-02	ESC	Tekko32 F4 (telemetria DShot)	6
P-03	Eliche	2 pale (dim. da confermare)	6
A-01	Flight controller	Pixhawk 6X (PX4, 3 IMU)	1
A-06	GPS / GNSS	CubePilot Here+ (su mast)	1
A-07	Base RTK	Stazione base	1
A-09	Cavo GPS↔Pixhawk	Auto-costruito, schermato (JST-GH)	1
E-01	Batteria	LiPo 4S 5600 mAh (XT60)	1
E-02	BEC / regolatore	Step-down 16,8 V → 5 V	1
C-01	Ricevitore RC	FrSky X8R	1
C-03	Telemetria	Holybro 433 MHz (TX/RX)	1 set
SW-01	Firmware FC	PX4	—
SW-02	Ground station	QGroundControl	—

Nota sulla configurazione hardware

Il flight controller attuale (Pixhawk 6X) è il risultato di una sostituzione hardware resa necessaria da un guasto del precedente Cube Black (Capitolo 6). Anche il cavo GPS è stato ricostruito con schermatura dedicata a seguito della diagnosi dei dropout `sensor_gps`. La BOM riflette quindi lo stato hardware consolidato a valle di questi interventi.

Capitolo 3

Configurazione: QGroundControl e firmware PX4

Una volta assemblato l'hardware, il velivolo deve essere reso *volabile*: occorre dire al firmware com'è fatto il drone, tarare i sensori e abilitare la registrazione dei dati. Tutte queste operazioni sono state svolte tramite la **ground station QGroundControl** (QGC), l'applicazione desktop che dialoga con il firmware **PX4** del Pixhawk 6X: permette di leggere e scrivere i *parametri* del veicolo, eseguire le calibrazioni e monitorare la telemetria in tempo reale.

Il collegamento tra QGC e il Pixhawk è sempre avvenuto attraverso il **link radio di telemetria a 433 MHz**, sia a banco sia in campo: la coppia di moduli Holybro fa da ponte seriale wireless tra il computer e il flight controller, e non è mai stato necessario collegare il drone via cavo USB. Questa scelta ha il vantaggio di lavorare sempre nella stessa configurazione con cui il drone vola, senza spostare cavi tra banco e campo.

Il capitolo ripercorre le configurazioni che hanno reso il drone operativo e strumentato per l'acquisizione dati. Diamo spazio anche alle **limitazioni e alle problematiche** incontrate: anche queste fanno parte a pieno titolo del lavoro svolto, perché documentano i limiti reali della piattaforma.

3.1 Calibrazioni e setup iniziale

Il *commissioning* (la prima messa a punto del veicolo) parte dalla selezione dell'**airframe**, cioè il modello geometrico con cui PX4 sa quanti motori ci sono e come sono disposti: nel nostro caso la configurazione esacottero **hexa_x** (sei motori a X). Da qui PX4 ricava la *matrice di mixing*, ossia come tradurre i comandi di assetto in spinte sui singoli motori.

Si procede poi con la **calibrazione dei sensori** — accelerometri, giroscopi e magnetometri — e con la **calibrazione della radio**, che associa le posizioni degli stick ai canali di comando. Infine si definiscono le *modalità di volo* assegnandole agli interruttori del radiocomando.

Per le prove a banco, in cui non si vuole (o non si può) avere un aggancio GPS, è stato abilitato l'**arming senza fix GPS** (`COM_ARM_WO_GPS = 1`) in modalità *Stabilized*; il parametro è stato riportato a 0 per i voli all'aperto, dove la posizione è indispensabile e armare senza GPS sarebbe pericoloso.

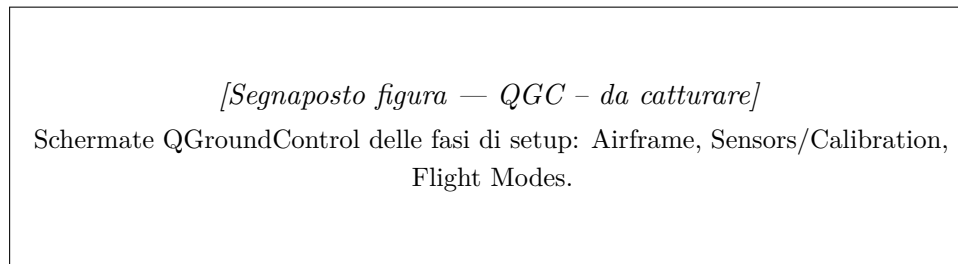


Figura 3.1: Schermate QGroundControl delle fasi di setup: Airframe, Sensors/Calibration, Flight Modes.

3.2 Migrazione del GPS da CAN a UART

Il modulo GNSS (u-blox NEO-M8P, integrato nel CubePilot Here+) era originariamente interfacciato al flight controller **Cube Black** tramite bus **DroneCAN**, un protocollo seriale a due fili condiviso tra più dispositivi. La sostituzione del controllore con il Pixhawk 6X, resa necessaria dal guasto del Cube Black (Capitolo 6), ha reso non più utilizzabile quel collegamento: non si disponeva di un cablaggio CAN compatibile con il nuovo controllore. Si è quindi optato per un collegamento su porta **UART** dedicata (la GPS2), adattando un cavo con un connettore realizzato su misura. La soluzione, oltre a rendere possibile l'integrazione, offre un vantaggio diagnostico: l'UART è un collegamento punto-punto, privo dell'overhead del bus e più agevole da analizzare a livello di flusso seriale. Lo stesso cavo è stato successivamente oggetto di un intervento migliorativo, documentato nel medesimo capitolo.

Subito dopo la migrazione il GPS non agganciava più alcun satellite: la console di QGC riportava **baudrate: 0** e **rate reading: 0 B/s**, segno che sulla porta non transitava alcun dato. La causa era duplice — un residuo della configurazione DroneCAN ancora attiva e un disallineamento tra la *porta logica* dichiarata nei parametri e la *porta fisica* a cui il cavo era collegato. La soluzione è nei parametri di Tabella 3.1, seguiti da un *cold start* all'aperto (primo aggancio da fermo, a cielo libero).

Tabella 3.1: Parametri risolutivi della migrazione GPS da DroneCAN a UART.

Parametro	Valore	Motivazione
UAVCAN_ENABLE	0	Disabilita DroneCAN, in conflitto col driver UART
GPS_1_CONFIG	202 (GPS 2)	Allinea la porta logica al cavo fisico (era 201 = GPS 1)
GPS_1_PROTOCOL	1 (UBX)	Protocollo binario u-blox

Nota di configurazione

Ogni cambio di firmware o riconfigurazione della porta GPS richiede di verificare la corrispondenza tra il parametro **GPS_x_CONFIG** e la porta fisica usata. Va inoltre segnalato che sul Pixhawk 6X non esistono i parametri **CAN_Px_DRIVER** citati in parte della documentazione: per disattivare il bus è sufficiente **UAVCAN_ENABLE = 0**.

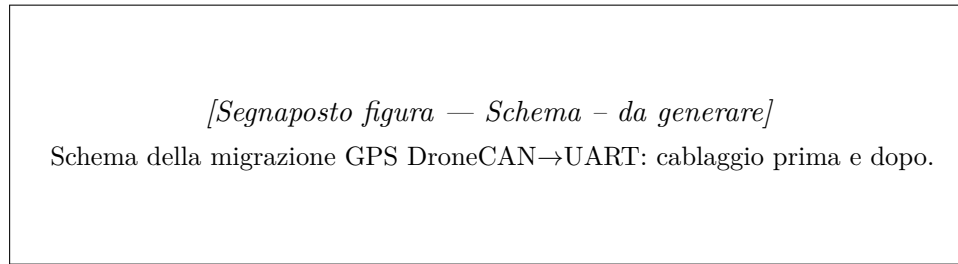


Figura 3.2: Schema della migrazione GPS DroneCAN→UART: cablaggio prima e dopo.

3.3 Power module e calibrazione della batteria

Il *power module* è il sensore che misura tensione e corrente assorbite dalla batteria e le pubblica al firmware. Inizialmente non pubblicava affatto il topic `battery_status` (lo stato risultava *never published*): il firmware non sapeva su quali ingressi analogici leggere i due valori. Assegnando i canali ADC corretti (`BAT1_V_CHANNEL = 16` per la tensione, `BAT1_I_CHANNEL = 17` per la corrente) e completando la calibrazione di tensione, la telemetria di tensione e corrente è diventata disponibile a ~ 5 Hz.

La taratura della batteria è tuttavia rimasta **problematica**. Il firmware continua a segnalare alcuni **parametri mancanti** nella configurazione del pacco e, di conseguenza, mostra in QGC un **livello di carica fisso**, non legato allo stato reale della batteria. Lo stesso si riflette nei log: la **percentuale di carica (SoC) registrata non è affidabile** e non rispecchia la scarica effettiva. Per questo motivo, durante la campagna di volo, l'autonomia residua è stata valutata in modo indiretto — sulla **tensione di pacco** e sul tempo di volo — e non sulla percentuale riportata dal firmware (vedi anche Capitolo 5). Tensione e corrente istantanee restano comunque corrette e utilizzabili.

3.4 Telemetria ESC su TELEM2

Gli ESC (*Electronic Speed Controller*, i regolatori che pilotano i motori) sono Tekko32 F4 con firmware AM32 e sono comandati in **DShot**, un protocollo digitale che invia i comandi di spinta senza le imprecisioni del vecchio segnale PWM analogico. Ogni ESC restituisce inoltre la propria **telemetria** (la cosiddetta telemetria KISS) su un filo dedicato; i sei fili sono uniti su un'unica linea seriale (TELEM2). Per evitare collisioni sul bus condiviso, il flight controller interroga gli ESC uno alla volta (*request-driven*): chiede il dato e attende la risposta, in sequenza.

La configurazione è riassunta in Tabella 3.2. Il risultato è il topic `esc_status` a ~ 65 Hz, con regime (RPM), tensione, corrente e temperatura di ciascuno dei sei motori — dati centrali per individuare squilibri ed eliche danneggiate (Capitolo 5).

Tabella 3.2: Parametri della telemetria ESC e del logging ad alta frequenza.

Parametro	Valore	Significato
DSHOT_CONFIG	600	Protocollo DShot600 di comando agli ESC
DSHOT_TEL_CFG	102 (TELEM2)	Porta seriale su cui leggere la telemetria KISS
SDLOG_PROFILE	849	Profilo di logging (vedi Cap. 4)
IMU_GYRO_RATEMAX	1000 Hz	Frequenza massima del giroscopio

3.5 Modalità di volo e failsafe

Nei test sono state usate due modalità di volo. In *Stabilized* il pilota comanda direttamente l'assetto (inclinazione) del drone, mentre la quota è gestita manualmente con il throttle; è la modalità usata per gli hover e per le prove con le pale. In *Position* (POSCTL) il velivolo mantiene attivamente la propria posizione usando il GPS, e a stick fermi resta sul punto; è la modalità usata per le traiettorie.

Un **failsafe** è una reazione automatica del firmware a una condizione anomala. Quello più rilevante per noi è il failsafe di **perdita di posizione**: se la stima di posizione diventa inaffidabile, PX4 può forzare un atterraggio automatico «alla cieca» (*blind-land*). A seguito degli eventi descritti nel Capitolo 6, abbiamo rivisto questo comportamento. Il punto chiave è che la decisione reale di blind-land nasce a livello del filtro di stima **EKF** (il filtro che fonde IMU, GPS e barometro per stimare assetto e posizione), tramite il parametro **EKF2_NOAID_TOUT**, e non solo nei parametri del *commander*: agire solo su questi ultimi non basta. La Tabella 3.3 riassume la configurazione adottata, orientata a concedere più tempo prima del failsafe e a ridurre la deriva orizzontale durante un eventuale evento.

Tabella 3.3: Configurazione del failsafe di perdita posizione e dei limiti di volo.

Parametro	Default	Applicato	Motivazione
EKF2_NOAID_TOUT	5 s	10 s	Ritarda xy_valid=false, vera causa del blind-land
COM_POSCTL_NAVL	Land	Altitude	Lascia gli stick orizzontali attivi al pilota
COM_POS_FS_EPH	5 m	10 m	Soglia di accuratezza orizzontale del commander
MPC_XY_VEL_MAX	12 m/s	5 m/s	Riduce la deriva orizzontale massima
MPC_TILTMAX_AIR	45°	25°	Limita inclinazione e velocità massima

3.6 Limitazioni e problematiche riscontrate

La configurazione ha avuto alcuni limiti che merita documentare:

- **RTK non utilizzabile**: la correzione RTK, che porterebbe la precisione di posizione a livello centimetrico, non è stata sfruttabile perché il driver GPS va in crash alla ricezione dei *fix* (Capitolo 6). La campagna è quindi stata condotta con il solo GPS standard (~80 cm di precisione).

- **Parametri non disponibili nella build PX4:** alcuni parametri di failsafe documentati nei manuali (`COM_POS_FS_DELAY`, `COM_POS_FS_EPV`) non esistono in questa versione del firmware; altri (`MPC_JERK_MAX`, `MPC_ACC_HOR`) non sono modificabili a mano perché vincolati automaticamente dal *trajectory shaping* (la generazione interna della traiettoria). In QGC gli slider *Flight Behavior* agiscono come comandi di alto livello su questi gruppi di parametri.
- **Override RC spurio sul canale yaw:** in un volo automatico la missione è stata interrotta da un *takeover* (passaggio forzato al controllo manuale) non voluto, innescato da un piccolo disturbo ($\sim 10\%$) sul canale yaw a stick fermi. Pur sotto la soglia nominale `COM_RC_STICK_OV`, il disturbo è bastato a far scattare l'override. Le mitigazioni possibili sono allargare la *deadzone* del canale, alzare la soglia o disabilitare l'override in modalità automatica.

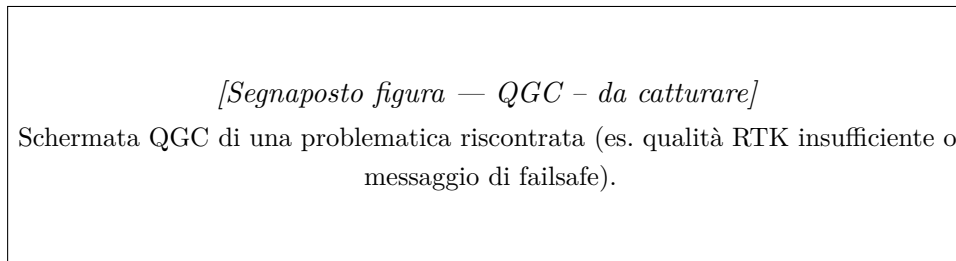


Figura 3.3: Schermata QGC di una problematica riscontrata (es. qualità RTK insufficiente o messaggio di failsafe).

Capitolo 4

Acquisizione dati

L'acquisizione dati è il cuore del progetto: l'arco di lavoro *configura* → *testa acquisendo dati* ha senso solo se il velivolo registra, a frequenza adeguata, le grandezze che permettono di valutare lo stato di salute del sistema. Questo capitolo descrive *cosa* viene loggato, *a che frequenza* e *con quale architettura*.

4.1 Il formato ULog di PX4

PX4 registra i dati in formato **ULog** (`.ulg`): un file binario auto-descrittivo che contiene la definizione dei topic, il valore di tutti i parametri del veicolo al momento dell'arming e lo stream dei messaggi *timestamped* (microsecondi dal boot). Viene creato un file per ogni ciclo arming→disarming, salvato su microSD in `/fs/microsd/log/<data>/<orario>.ulg`. La lettura off-line avviene con la libreria Python `pyulog` (estrazione topic, frequenze, CSV) e con strumenti come PX4 Flight Review, PlotJuggler e — per il replay 3D — la pipeline Foxglove descritta nel Capitolo 8.

4.2 Architettura uORB e pipeline di logging

In PX4 i driver dei sensori pubblicano messaggi sul bus interno **uORB** (micro Object Request Broker), un sistema publish/subscribe. Il modulo **logger non campiona**: si limita a sottoscrivere i topic uORB e a scriverli su SD, perciò la frequenza di logging coincide con la frequenza di pubblicazione del topic. Un volo standard espone circa **147 topic** uORB. La Figura 4.1 sintetizza la catena dei dati.

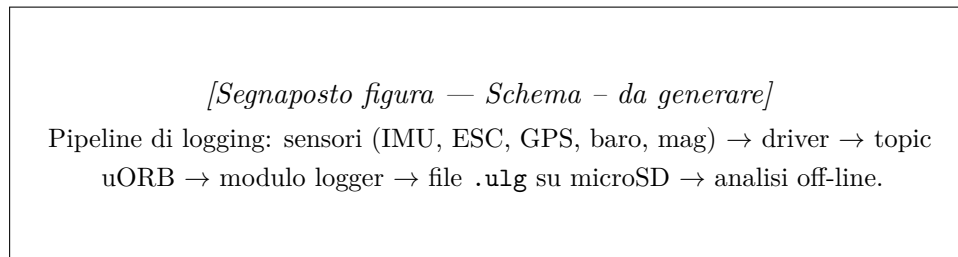


Figura 4.1: Pipeline di logging: sensori (IMU, ESC, GPS, baro, mag) → driver → topic uORB → modulo logger → file `.ulg` su microSD → analisi off-line.

4.3 Profilo di logging `SDLOG_PROFILE = 849`

La selezione di *cosa* loggare è governata dalla bitmask `SDLOG_PROFILE`. Il valore adottato (849) attiva, oltre al profilo di default, l'alta frequenza, il confronto fra le tre IMU e i FIFO grezzi di giroscopio e accelerometro — proprio i dati necessari all'analisi vibrazionale (Tabella 4.1).

Tabella 4.1: Decodifica della bitmask `SDLOG_PROFILE = 849`.

Bit	Valore	Contenuto
0	1	Default (stato veicolo, GPS, modalità, batteria)
4	16	High rate (IMU a piena frequenza)
6	64	Sensor comparison (3 IMU separate)
8	256	Raw gyro FIFO
9	512	Raw accel FIFO
Totale = 849		

4.4 Frequenze di acquisizione

La Tabella 4.2 riepiloga i topic più rilevanti per la manutenzione preventiva con le frequenze *misurate* sul log di riferimento (2026-04-28/10_28_38.u1g, ~166 MB, ~4 min). Si distinguono tre fasce: i **FIFO inerziali** a ~1 kHz (per la FFT delle vibrazioni), i topic di controllo e assetto a centinaia di Hz, e la telemetria «lenta» (batteria, GPS) a pochi Hz. L'elenco esteso è mantenuto in `plot/DATI_LOG.md` e richiamato in Appendice A.

Tabella 4.2: Topic uORB principali e frequenze misurate (log di riferimento).

Topic	Freq.	Uso per la manutenzione
<code>sensor_gyro_fifo</code>	~983 Hz	FFT vibrazioni: squilibri elica/cuscinetti
<code>sensor_accel_fifo</code>	~957 Hz	FFT accelerazioni, squilibri strutturali
<code>actuator_motors</code>	~889 Hz	Comandi normalizzati ai 6 motori
<code>vehicle_attitude</code>	~144 Hz	Qualità del controllo d'assetto
<code>sensor_combined</code>	~144 Hz	IMU combinata (gyro+accel)
<code>esc_status</code>	~65 Hz	RPM, corrente, temperatura per ESC
<code>manual_control_setpoint</code>	~43 Hz	Input stick del pilota
<code>vehicle_local_position</code>	~10 Hz	Posizione stimata (NED)
<code>hover_thrust_estimate</code>	~10 Hz	Deriva spinta di hover (peso/motori)
<code>battery_status</code>	~5 Hz	Tensione e corrente (SoC non affidabile)
<code>vehicle_imu_status</code>	~1 Hz	Metriche vibrazione e clip per IMU
<code>failure_detector_status</code>	~2 Hz	Squilibrio elica, motori in failure
<code>vehicle_gps_position</code>	~4,5 Hz	Qualità fix GPS

4.4.1 FIFO inerziali ad alta frequenza

I topic `sensor_gyro_fifo` e `sensor_accel_fifo` contengono *burst* di campioni per ogni messaggio loggato (frequenza effettiva ~1 kHz). Questa risoluzione consente l'analisi FFT delle

vibrazioni: la firma spettrale di un'elica accorciata o di un cuscinetto usurato è visibile come picco alla frequenza di rotazione e alle sue armoniche. È la base dell'analisi del Capitolo 7.

4.4.2 Grandezze chiave per la manutenzione preventiva

Tra i 147 topic, alcuni sono particolarmente diagnostici: `vehicle_imu_status.accel_vibration_metric` (livello di vibrazione), `failure_detector_status.imbalanced_prop_metric` (squilibrio elica stimato da PX4), `esc_status` (squilibri di RPM/corrente tra motori) e `battery_status` (tensione e corrente di pacco; le stime derivate come SoC e SoH richiedono però una configurazione della batteria che nel nostro caso è rimasta incompleta, Capitolo 3). Questi indicatori permettono di intercettare la degradazione *prima* che diventi guasto.

4.5 Qualità del logging e dropout

La capacità di scrittura della microSD è il collo di bottiglia: un log a banco da 28 s (`16_02_24.ulg`) ha registrato 9 *dropout* per 0,4 s complessivi, che colpiscono soprattutto i FIFO ad alta frequenza. La mitigazione più efficace è una **microSD veloce** (classe UHS-I U3/V30, ≥ 30 MB/s sostenuti); in subordine si può snellire `SDLOG_PROFILE` rimuovendo i bit non utilizzati. Questa attenzione è essa stessa una voce di manutenzione preventiva, perché un log con buchi compromette le analisi a posteriori.

Capitolo 5

Campagna di volo

La campagna di volo è la fase in cui il drone, ormai configurato e strumentato, viene fatto volare per *acquisire dati*. L'obiettivo è studiare in modo sistematico come il **danneggiamento di una pala** si rifletta sul comportamento dell'esacottero e, soprattutto, sui dati registrati a bordo. Questo capitolo descrive la metodologia adottata, i tipi di volo eseguiti e il quadro complessivo delle sessioni.

5.1 Metodologia

L'idea sperimentale è semplice: **isolare una variabile alla volta** e confrontare i dati con una condizione di riferimento. Si parte da una *baseline* con tutte le eliche integre e si introduce poi, in modo controllato e ripetibile, un danno noto su una sola pala.

Baseline (pale sane) tutte e sei le eliche integre. Per smascherare un eventuale *bias* di montaggio (un piccolo squilibrio dovuto al modo in cui le pale sono fissate), a ogni volo baseline si scambia la posizione di due pale.

Pala danneggiata una pala viene accorciata di una percentuale nota della sua lunghezza e montata a turno su ciascuno dei sei motori (M1...M6). L'accorciamento riproduce in modo ripetibile l'effetto di un'elica usurata o scheggiata: riduce la spinta di quel rotore e ne sbilancia la massa, generando vibrazione e costringendo il controllore a compensare.

Ogni volo segue lo stesso schema — durata di circa un minuto e registrazione completa su scheda SD — per rendere i log confrontabili tra loro.

Che cosa significa «pala 5 %» e «pala 10 %»

La pala di prova viene accorciata, rispetto alla lunghezza nominale, del **5 %** oppure del **10 %**. Maggiore l'accorciamento, maggiore lo squilibrio aerodinamico e di massa: il 5 % rappresenta un danno *lieve* (visibile solo nei picchi delle metriche), il 10 % un danno *marcato* (le metriche di vibrazione raddoppiano, vedi Capitolo 7). Le prove al 15 % sono state sospese per decisione del docente.

5.2 Numerazione dei motori e tipi di traiettoria

Per attribuire un effetto al motore giusto serve una convenzione di numerazione non ambigua. Si adotta quella del layout PX4 `hexa_x` (Figura 5.1): le coppie di motori diagonalmente opposti sono $M1 \leftrightarrow M2$, $M3 \leftrightarrow M6$ e $M4 \leftrightarrow M5$; le rotazioni sono antiorarie (CCW) su $M1$, $M2$, $M5$ e orarie (CW) su $M3$, $M4$, $M6$.

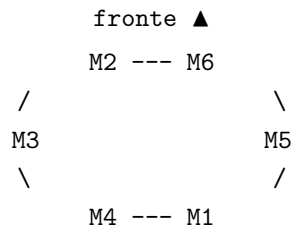


Figura 5.1: Layout e numerazione dei motori (PX4 `hexa_x`, vista dall'alto). Schema provvisorio testuale, da sostituire con figura vettoriale.

Sono state usate due geometrie di volo, scelte per sollecitare il sistema in modi diversi:

Hover stazionario il drone mantiene la quota fermo sul posto, in modalità *Stabilized* con controllo manuale. È la condizione più «pulita» per misurare vibrazione e squilibrio, perché il velivolo non manovra. Comprende i Set A–C.

Quadrato il drone percorre un quadrato di lato ~ 8 m a una quota di ~ 8 m. Qui la pala danneggiata deve lavorare anche durante le manovre, sollecitando il controllore. Comprende i Set D–E.

I quadrati sono stati eseguiti in **modalità di volo autonoma**: la traiettoria non viene pilotata a mano, ma definita in anticipo come piano di volo e seguita automaticamente dal firmware. Concretamente, nella vista *Plan* di QGroundControl si dispone una sequenza di *waypoint* ai quattro vertici del quadrato, preceduti da un *waypoint* di decollo e chiusi da uno di atterraggio; per ogni *waypoint* si fissano quota e velocità di crociera. Il piano viene poi caricato sul veicolo, che lo esegue in modalità *Mission* (AUTO). Eseguire i quadrati in autonomo — anziché a mano — garantisce che la **traiettoria sia ripetibile** tra un volo e l'altro, eliminando la variabilità del pilota e rendendo confrontabili le prove a parità di danno.

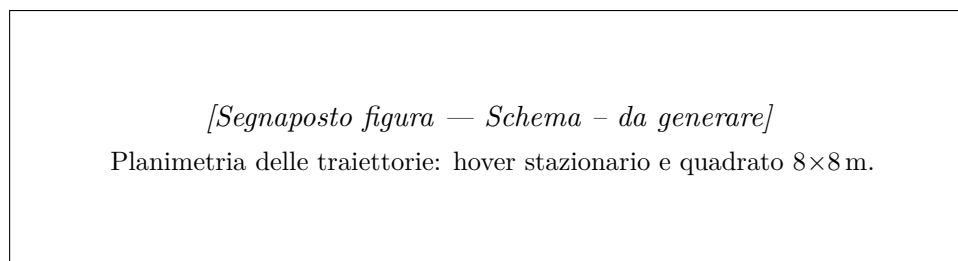


Figura 5.2: Planimetria delle traiettorie: hover stazionario e quadrato 8×8 m.

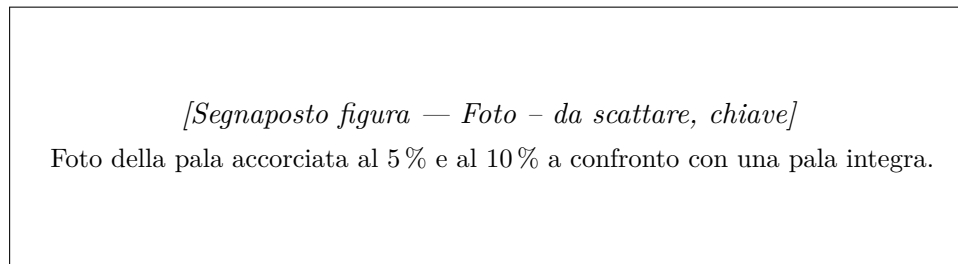


Figura 5.3: Foto della pala accorciata al 5 % e al 10 % a confronto con una pala integra.

5.3 Quadro delle sessioni e dei Set

La Tabella 5.1 riepiloga i Set previsti e il loro stato finale. La campagna è considerata **conclusa**: sono stati completati gli hover (Set A–C) e i quadrati (Set D–E), più un blocco aggiuntivo con **payload** (Set P) non previsto nel piano originale. Le tabelle volo-per-volo, con la mappatura tra file di log e prova, sono raccolte in Appendice A.

Tabella 5.1: Riepilogo dei Set di volo (stato finale della campagna).

Set	Tipo di volo	Eseguiti
A	Hover, pale sane (swap posizione)	6/6
B	Hover, pala 5 % ($\times 6$ motori)	6/6
C	Hover, pala 10 % ($\times 6$ motori)	6/6
D	Quadrato 8×8 m, pala 5 %	13/14
E	Quadrato 8×8 m, pala 10 %	14/14
F	Traiettoria casuale, pala 5 %	non eseguito
G	Traiettoria casuale, pala 10 %	non eseguito
P	Payload (2 liberi sani + 3 quadrati M2)	5
H/I/L	Prove al 15 %	<i>sospese (docente)</i>

5.4 Condizioni operative e voli scartati

Le sessioni in campo si sono svolte con vento da **15 a 25 km/h**, una condizione non ideale che ha richiesto correzioni continue del pilota e che si riflette nella variabilità dei dati. Il fattore più limitante è però stato l'**autonomia della batteria**: in più occasioni la seconda metà di una sessione ha richiesto ripetizioni (atterraggi manuali, voli annullati) per pacco ormai scarico, soprattutto sulle prove dei motori M4 e M6. Poiché la percentuale di carica riportata dal firmware non è affidabile (Capitolo 3), lo stato del pacco è stato sorvegliato sulla **tensione** e sul tempo di volo.

Alcuni voli sono stati deliberatamente **scartati** per cause note — un atterraggio d'emergenza per batteria scarica e un decollo fallito per errore del magnetometro. Tenere traccia di questi episodi è essenziale per non confondere un artefatto operativo con un reale effetto del danno alla pala.

Bias strutturale da considerare

L'analisi dei voli baseline ha evidenziato un **bias di montaggio** costante e indipendente dal danno (M3/M6 leggermente sotto-comandati, M4/M5 sopra-comandati, nell'ordine del $\pm 6\%$). È importante caratterizzarlo per non confonderlo con la firma del danno alla pala (Capitolo 7).

Capitolo 6

Casi diagnostici

La messa in funzione del drone ha attraversato diversi guasti e anomalie. Questo capitolo li raccoglie raggruppandoli per **sottosistema**, anziché in stretto ordine cronologico: ciascun caso è stato l'occasione per una diagnosi sistematica e per un'azione correttiva, e ogni soluzione adottata si è tradotta in una buona pratica per i voli successivi. Si parte dal flight controller, si passa al collegamento GPS (il sottosistema che ha richiesto più lavoro), si discute la mancata disponibilità dell'RTK e il quasi-incidente da perdita di posizione che ne è derivato, e si chiude con l'analisi di un incidente in volo.

6.1 Sostituzione del flight controller

Il flight controller iniziale era un **CubePilot Cube Black** (basato sul microcontrollore STM32F427). Dopo aver impostato `IMU_GYRO_RATEMAX` a una frequenza troppo alta per quel processore, il modulo ha smesso di connettersi al computer: la porta seriale appariva per pochi secondi e poi spariva. Il parametro corrotto risiede nella **memoria flash interna** del controller, non sulla scheda SD; per questo reflashare il firmware — operazione che sovrascrive il codice ma non l'area dei parametri — non risolveva. Tutti i tentativi software (cambio cavo, script di avvio con `param reset_all`, recovery da bootloader) sono falliti, perché il blocco avviene durante l'inizializzazione degli IMU, *prima* che la console diventi disponibile. La soluzione è stata la **sostituzione hardware** con il Pixhawk 6X, basato sul più potente STM32H7.

Lezione appresa

Il valore di `IMU_GYRO_RATEMAX` va scelto in base al microcontrollore: sullo STM32F427 del Cube Black non conviene superare ~ 2000 Hz, mentre le frequenze più alte richiedono uno STM32H7 (come quello del Pixhawk 6X). È inoltre utile disporre di un programmatore SWD (ST-Link) per il recupero a basso livello in caso di blocco.

6.2 Messa a punto del collegamento GPS

6.2.1 Mancato aggancio dopo la migrazione a UART

Il primo problema sul GPS si è presentato subito dopo lo spostamento del modulo dal bus DroneCAN alla porta UART: il ricevitore non agganciava alcun satellite. La causa (un residuo di configurazione DroneCAN e un disallineamento tra porta logica e porta fisica) e la relativa soluzione sono descritte nel Capitolo 3.

6.2.2 Dropout del GPS in volo: diagnosi e soluzione

Risolto l'aggancio, durante i voli all'aperto sono comparsi ripetuti **dropout** del topic `sensor_gps`: in QGC si leggevano messaggi di «GPS failsafe» e «GNSS data fusion stopped», con conseguente retrocessione della modalità di volo. Il fenomeno era intermittente e si manifestava in volo, non a terra.

Per diagnosticarlo si è abilitato `GPS_DUMP_COMM = 1`, che pubblica il flusso seriale grezzo del ricevitore nel topic `gps_dump`: questo ha permesso di analizzare il bytestream UBX (il protocollo binario u-blox) frame-per-frame. Misurando la frazione di byte corrotti in ricezione (il *bit error rate*, BER) si è visto che a motori al minimo il link è praticamente pulito (BER 0,2%), mentre in volo il BER diventa **erratico**, oscillando tra 0,2% e 49% *a parità di regime motori* (Tabella 6.1). È un dettaglio cruciale: il volo 13_51_13, pur girando al regime di lift più alto della sessione (39 597 sum-RPM), resta pulito allo 0,2%, mentre 13_51_37, a regime inferiore, collassa al 49%. Il BER non scala dunque con la potenza erogata.

Tabella 6.1: BER del link GPS (byte fuori frame in ricezione). A motori fermi il link è pulito; in volo il BER diventa erratico *a parità di regime*: si confronti 13_51_13 (regime massimo, link pulito) con 13_51_37 (regime inferiore, BER 49%). L'assenza di correlazione con la potenza esclude un accoppiamento EMI dai cavi motore.

Log	Durata	Sum-RPM max	Garbage RX	Reinit driver
13_50_41 (idle)	12 s	4 270	0,2 %	0
13_51_13 (volo)	20 s	39 597	0,2 %	0
13_50_52 (volo)	12 s	26 826	14,8 %	1
13_51_37 (volo)	96 s	32 841	49,0 %	2
13_40_57 (volo)	344 s	47 740	15,5 %	6

La diagnosi ha escluso le cause alternative (perdita di satelliti, brownout dell'alimentazione 5 V, reset del modulo comandato da PX4, firmware, baud rate errato) e — punto su cui vale la pena soffermarsi — ha **falsificato anche l'ipotesi delle interferenze elettromagnetiche (EMI)**. Se il disturbo fosse irradiato dallo stadio motori e accoppiato sulla linea seriale, il BER dovrebbe crescere con la corrente nei cavi di potenza; invece il volo 13_51_13 è pulito proprio al regime di lift più alto, e le metriche RF del ricevitore (`noise_per_ms`, `jamming_indicator`, `agc`) restano costanti prima, durante e dopo i dropout. La firma dei dati — BER che salta tra 0,2% e 49% a parità di regime — è invece quella di un **contatto meccanico intermittente nel cavo auto-costruito**, che si apre e richiude sotto le vibrazioni del telaio a seconda di come il cavo si posa.

La soluzione è stata **ricostruire il cavo, ri-crimparlo e schermarlo**: una calza in rame stagnato attorno ai conduttori, con *drain wire* (filo di drenaggio) saldato a massa solo dal lato Pixhawk, hot-glue sui connettori JST-GH e instradamento separato dai cavi motore. La schermatura rimuove un eventuale contributo EMI residuo, ma è soprattutto la ri-crimpatura a eliminare il contatto marginale, che era la causa dominante. Il volo di accettazione ha riprodotto proprio le condizioni che prima facevano collassare il link, con esito positivo: il BER è crollato dal **49 % allo 0,24 %** a parità di regime motori, con **zero reinizializzazioni** del driver e nessun buco del topic `sensor_gps` superiore a 2s. La causa radice è quindi rimossa, e il cavo schermato è entrato nella BOM come configurazione consolidata.

La rimozione di questo guasto ha però messo in luce un **secondo modo di guasto, indipendente**, che il cavo difettoso prima mascherava: con il cavo sano, i voli in GPS puro del 04/06 non mostrano più alcun reinit, ma quelli in RTK sì. Le due cause sono dunque *in sequenza* — prima il cavo, poi l'RTK — e quest'ultima è oggetto della sezione seguente.

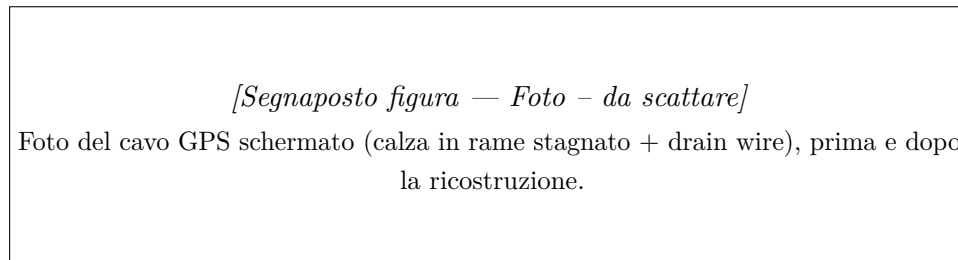


Figura 6.1: Foto del cavo GPS schermato (calza in rame stagnato + drain wire), prima e dopo la ricostruzione.

6.3 RTK non utilizzabile: voli con il solo GPS

Il sistema prevede la correzione **RTK** (*Real-Time Kinematic*), che sfrutta una stazione base a terra per portare la precisione di posizione dai ~80 cm del GPS standard al livello centimetrico. Nella pratica, però, non è stato possibile utilizzarla: **quando il ricevitore inizia a ricevere le correzioni e ad agganciare un *fix* RTK, il driver GPS di PX4 va in crash** e si re-inizializza, interrompendo per alcuni secondi la pubblicazione del topic `sensor_gps`. Il malfunzionamento si manifesta proprio nel momento in cui l'RTK porterebbe il suo vantaggio, rendendo la correzione di fatto inutilizzabile (la diagnosi è documentata in dettaglio nel troubleshooting di progetto).

Si è quindi deciso di condurre **l'intera campagna con il solo GPS standard**. Per il tipo di prove svolte questa scelta non è penalizzante: la precisione di ~80 cm è più che sufficiente, perché l'oggetto dello studio è l'effetto del danno alla pala su vibrazioni, RPM e comandi ai motori — grandezze che non dipendono dalla precisione assoluta di posizione — e la ripetibilità della traiettoria è comunque garantita dal piano di volo autonomo.

6.3.1 Evidenza A/B: la preconditione è il modo RTK

Resta da stabilire *che cosa* esattamente inneschi il crash del driver: una singola correzione RTCM malformata, oppure il modo RTK in sé (cioè il fatto stesso di iniettare il flusso RTCM

nel ricevitore). Una sessione del 04/06 ha offerto un confronto **A/B quasi sperimentale**: tre voli con la *stessa* traiettoria quadrata, eseguiti di seguito, in cui l'unica variabile che cambia è il modo GPS (Tabella 6.2).

Tabella 6.2: Confronto A/B sulla stessa missione quadrata (04/06): a parità di traiettoria cambia solo il modo GPS. Il *fix* RTK è identificato dalla congiunzione di tre segnali (`fix_type`≥5, RTCM iniettato a bordo, `eph` a livello centimetrico).

Volo	Modo	eph	RTCM	Esito
14_06_22	RTK reale	3,5 cm	3,1/s	21 s, 0 reinit, sano
14_07_19	RTK reale	1,9 cm	3,1/s	1 reinit → 28,4 s di outage → failsafe ALTCTL
14_13_15	GPS puro	54 cm	0	124 s, 0 perdite di GPS

Il dato decisivo è che il volo problematico (14_07_19) ha perso il GPS con il flusso di correzioni **sano fino all'ultimo campione** (CRC RTCM a zero, rate stabile, `eph` di 1,7 cm sull'ultimo *fix* valido): non è quindi una correzione corrotta a innescare lo stallo, ma il **modo RTK stesso**. La controprova è il volo gemello in GPS puro (14_13_15), che ha ripetuto la medesima missione per oltre due minuti senza alcuna interruzione. Questa è la ragione per cui la campagna è stata condotta deliberatamente in GPS standard: oltre a non servire la precisione centimetrica, volare senza RTK *evita il modo di guasto*.

Come si distingue GPS puro da RTK reale in un log

Nessun campo da solo è affidabile. La firma robusta di RTK realmente attivo è la **congiunzione** di tre segnali indipendenti in `sensor_gps`: `fix_type`≥5 (5 = RTK-float, 6 = RTK-fixed), `rtcm_injection_rate`>0 (PX4 sta iniettando correzioni) ed `eph` che scende sotto ~10 cm (precisione che solo l'RTK fornisce). Il campo `rtcm_msg_used` è invece **inutilizzabile** su questo modulo: vale sempre 0 («unknown») anche nei voli con `eph` di 2 cm, perché il driver non lo popola.

Un affinamento importante del modello di guasto: la **metrica di gravità non è il numero di reinizializzazioni**, ma la **durata dell'outage**. In 14_07_19 una *singola* reinit ha prodotto 28,4 s di buco di navigazione e un failsafe operativo, mentre altrove una reinit isolata si è risolta in un gap di 2 s del tutto benigno. La reinizializzazione del driver è il *recupero*, non il grilletto: ciò che conta è quanto a lungo l'EKF resta senza aggiornamenti di posizione — ed è proprio questo che governa il failsafe discusso nella sezione seguente.

6.4 Quasi-incidente da perdita GPS e failsafe a due livelli

Prima di ricostruire il cavo, due voli del 26/05 (10_45_41.ulg e 10_54_52.ulg) hanno mostrato la **conseguenza più pericolosa** di un dropout GPS: non il buco di dati in sé, ma la **reazione del failsafe**. In entrambi i voli la pubblicazione di `sensor_gps` si è interrotta per un intervallo **quasi identico** — 21,605 s e 21,609 s, una coincidenza al centesimo che indica un comportamento deterministico del driver, non un guasto stocastico. Durante il buco il drone

è finito in deriva orizzontale incontrollata e il pilota ha dovuto contrastarlo manualmente, generando i picchi di corrente (fino a ~ 47 A) che si osservano nei log come una classica **PIO**.

L'aspetto sorprendente emerso applicando le contromisure è che il parametro «ovvio» (`COM_POSCTL_NAVL`, la policy del *commander* alla perdita di posizione) **era già impostato in modo conservativo**, eppure il *blind-land* è scattato lo stesso. La ragione è che PX4 ha **due livelli di failsafe** sovrapposti sulla perdita di posizione (Tabella 6.3), e il livello basso scavalca quello alto.

Tabella 6.3: I due livelli di failsafe sulla perdita di posizione. Il livello basso (`mc_pos_control`) interviene per primo e scavalca la policy del *commander*.

Livello	Modulo	Logica e parametro
Alto	<code>commander</code>	«La posizione è persa, applico la policy»: <code>COM_POSCTL_NAVL</code> , <code>COM_POS_FS_EPH</code>
Basso	<code>mc_pos_control</code>	«Non ricevo setpoint validi, atterro alla cieca»: <i>blind-land</i> verticale <i>hard-coded</i>

Il vero «rubinetto» che decide *quanto a lungo* l'EKF tollera l'assenza di GPS prima di dichiarare `xy_valid=false` — e quindi di far scattare il *blind-land* di `mc_pos_control` — è `EKF2_NOAID_TOUT`, non i parametri del *commander*. È per questo che la configurazione del failsafe (Tabella 3.3, Capitolo 3) agisce *prima* su quel parametro, portandolo al massimo consentito dal firmware (10 s, dal default di 5 s). Va osservato un **limite residuo**: poiché il gap osservato è di $\sim 21,6$ s e il *cap* firmware è 10 s, resta una finestra di $\sim 11,6$ s in cui il failsafe scatta comunque. Le mitigazioni di contorno — velocità orizzontale e tilt massimo ridotti — servono proprio a **dimezzare la deriva** durante quella finestra scoperta, mentre la rete di sicurezza finale resta procedurale: il pilota è istruito a passare immediatamente in *Stabilized*, che bypassa l'intero stack di failsafe e restituisce controllo manuale puro.

Lezione appresa

In PX4 il failsafe di perdita posizione non si governa dal solo *commander*: il controllore di basso livello (`mc_pos_control`) ha un *blind-land* *hard-coded* che interviene per primo. Il parametro realmente determinante è `EKF2_NOAID_TOUT`; agire sui soli parametri *commander* dà una falsa sensazione di sicurezza.

6.5 Analisi forense dell'incidente in volo

In una delle sessioni si è verificato uno **schianto**, analizzato in dettaglio a partire dal log (`11_14_44.ulg`). Il drone, in decollo automatico a ~ 3 m, si è capovolto dopo circa 13 s. La ricostruzione (Figure 6.2 e 6.3) individua una causa **procedurale**, non un guasto hardware: al passaggio dal modo automatico a `POSCTL`, lo stick del throttle era a fondo corsa ($-1,0$), interpretato da PX4 come comando di *discesa massima*. Il tentativo di recupero del pilota ha innescato una *PIO* (*Pilot Induced Oscillation*, un'oscillazione che si auto-alimenta a causa dei comandi del pilota) divergente fino al ribaltamento; il *commander* è poi passato autonomamente

ad AUTO_LAND con il drone già capovolto, e l'impatto è avvenuto in caduta libera ($v_z = +4,95$ m/s). Tutti gli indicatori hardware (batteria, RPM, vibrazioni, EKF) erano nominali fino all'urto.

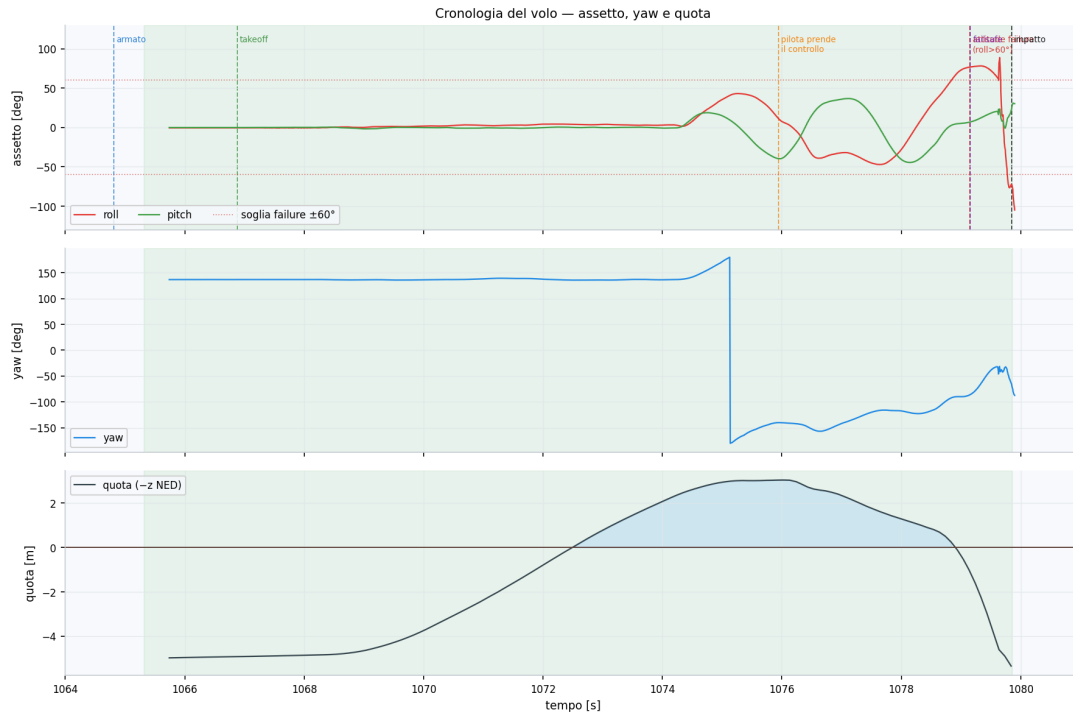


Figura 6.2: Cronologia dell'incidente: roll/pitch/yaw e quota con gli eventi annotati (takeover, AUTO_LAND, impatto).

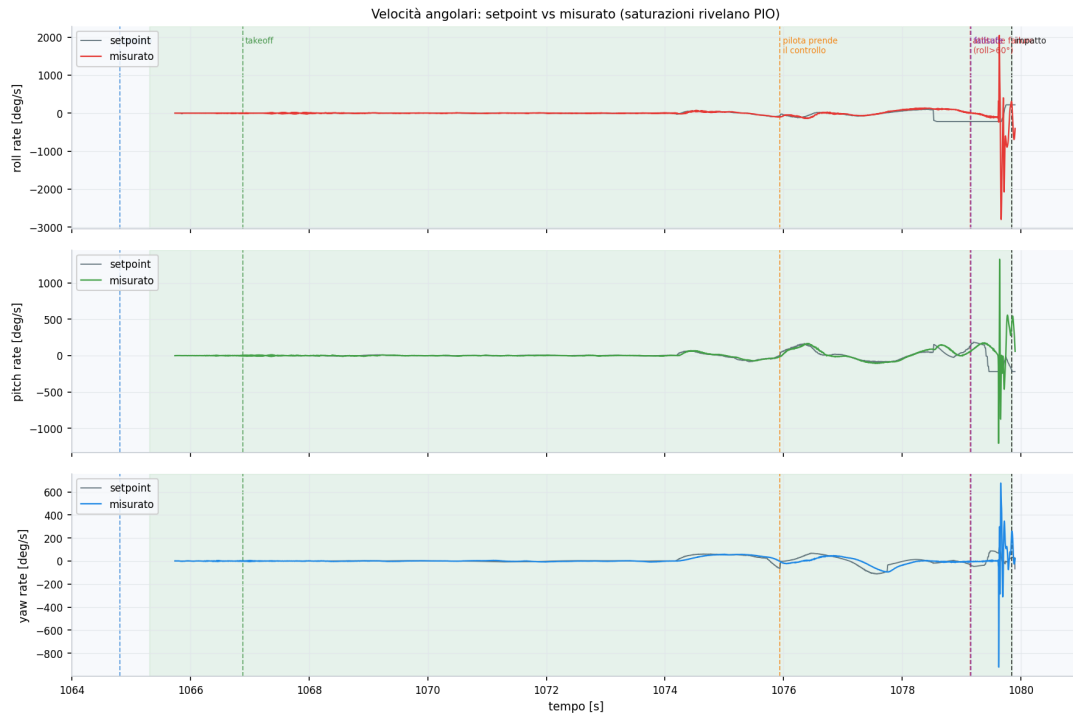


Figura 6.3: Dinamica angolare: rate setpoint vs misurato sui tre assi. La saturazione del rate ($\pm 220^\circ/\text{s}$) è la firma della PIO.

Da questa analisi discendono alcune precauzioni operative concrete: portare lo stick del throttle in posizione neutra *prima* di ogni cambio di modo verso POSCTL, preferire ALTCTL come modalità di transizione, rivedere il tuning del controllore di rate e verificare l'integrità strutturale dopo ogni impatto.

Capitolo 7

Osservazioni sui dati

Capitolo non richiesto dal progetto

Le analisi quantitative sui dati non erano un obiettivo del progetto (focalizzato su messa in funzione e acquisizione). Si riportano qui in forma sintetica perché mostrano che i dati acquisiti sono effettivamente *utili* alla diagnosi dello stato di salute, chiudendo il filo conduttore della relazione.

7.1 Metodo

Per ogni log, sull'intervallo armato più lungo, sono state estratte metriche scalari (vibrazione accelerometro e giroscopio, `imbalanced_prop_metric`, hover thrust) e metriche per-motore (RPM e comando normalizzato). Per queste ultime si usa il **rapporto** tra il valore medio del motore i e la media degli altri cinque: un valore > 1 indica che quel motore lavora più del gruppo. Il confronto è fra Set A (baseline), Set B (5%) e Set C (10%) in hover.

7.2 Vibrazione e squilibrio: baseline / 5 % / 10 %

La Tabella 7.1 sintetizza il confronto trasversale. Due risultati spiccano: (i) il danno al 5% è **quasi invisibile nelle medie** e vive solo nei picchi; (ii) al 10% **tutte le metriche di vibrazione raddoppiano**. La metrica con il miglior rapporto segnale/rumore è `imbalanced_prop_metric` ($\times 2$ al 5%, $\times 6$ al 10%).

Tabella 7.1: Metriche aggregate in hover: baseline vs pala 5% vs pala 10% (su M1).

Metrica	A (sane)	B (5 %)	C.1 (10 %)
<code>accel_vib</code> mean (m/s^2)	2,14	2,14	3,72
<code>accel_vib</code> max	3,45	4,03	7,74
<code>gyro_vib</code> mean (rad/s)	0,255	0,232	0,393
<code>gyro_vib</code> max	0,387	0,442	0,836
<code>imbalanced_prop</code> p95	-0,063	-0,134	-0,358
<code>hover_thrust</code> mean	0,525	0,513	0,555

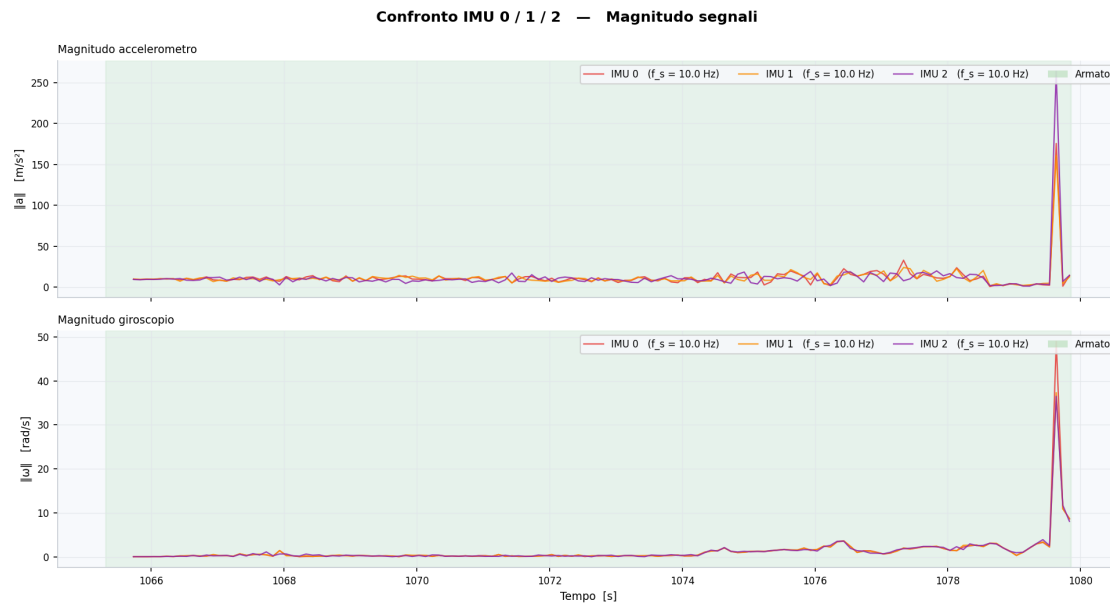


Figura 7.1: Confronto delle firme vibrazionali tra le configurazioni di pala.

7.3 Progressione monotona su M1

Su M1 — l'unico motore con tutti e tre i livelli di danno — la firma è **monotona e pulita** sia per il comando sia per gli RPM: il rapporto di comando passa da 0,975 (sano) a 0,995 (5 %) a 1,014 (10 %). Il mixer porta il motore dal sotto-comando strutturale al sopra-comando per compensare la pala mancante. La progressione quasi lineare suggerisce che questo rapporto possa fungere da **indicatore proxy del livello di danno** nell'intervallo 0–10 %.

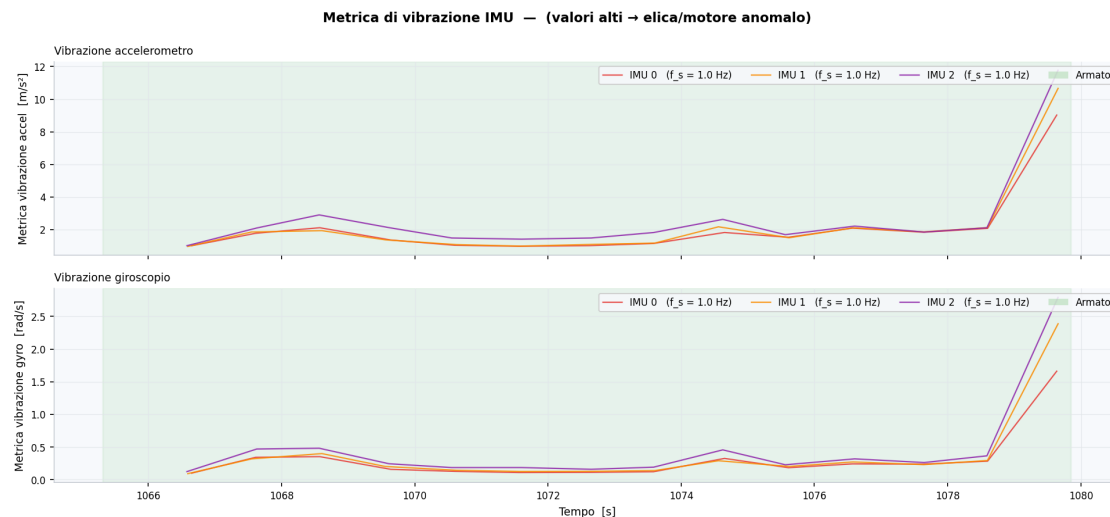


Figura 7.2: Andamento temporale della metrica di vibrazione durante l'hover.

7.4 Metriche raccomandate

In ordine di rapporto segnale/rumore osservato nel range 0–10 %: `imbalanced_prop_metric` p95, `accel_vib max`, `gyro_vib max` e il rapporto comando/RPM del motore danneggiato (utile

per stimare il *livello* di danno). Sono invece sconsigliate come metriche primarie **hover_thrust** e le *medie* di vibrazione, dominate dalla scarica batteria e dalla variabilità inter-volo.

Capitolo 8

Foxglove e interfaccia di visualizzazione

Contributo extra

Lo strumento descritto qui non era previsto dal progetto: è stato realizzato per comodità di analisi (in particolare per l'indagine forense dell'incidente del 26/05) e si è rivelato utile per tutta la campagna. È presentato in fondo come contributo aggiuntivo.

8.1 Motivazione

Gli script matplotlib (cartella `plot/`) sono ottimi per analisi mirate, ma non danno una visione *spazio-temporale* del volo. Per ricostruire visivamente l'incidente serviva un replay 3D sincronizzato con la telemetria. L'estensione PX4 ufficiale di Foxglove si è rivelata inaffidabile sui nostri log (alcuni topic, tra cui `vehicle_attitude`, non venivano convertiti, lasciando vuoto l'albero delle trasformate). La soluzione è stata una pipeline custom che precompila tutto il necessario nel file di output.

8.2 Pipeline `.ulg` → `.mcap`

Lo script `foxglove/ulog_to_mcap.py` converte il log PX4 in un file `.mcap` **autoconsistente** (nessun plugin richiesto in Foxglove). Oltre a copiare i 147 topic uORB originali, lo script inietta i dati derivati necessari al replay 3D:

- le trasformate `/tf` corpo (`local_origin`→`base_link`) con conversione di coordinate NED→ENU e FRD→FLU;
- le trasformate delle sei eliche, con angolo integrato dagli RPM di `esc_status` e fattore di scala visivo (per evitare l'aliasing *wagon-wheel*);
- topic interpretativi (`attitude_euler` in gradi, `flight_state` con quota positivo-up e nomi leggibili dei modi di volo) e i `logged_messages` del firmware.

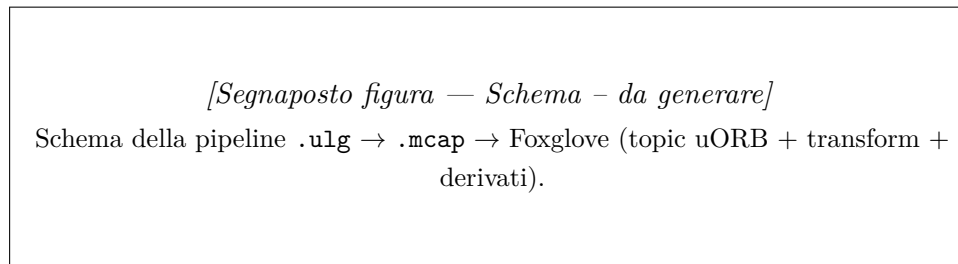


Figura 8.1: Schema della pipeline `.ulg` $\rightarrow$ `.mcap` $\rightarrow$ Foxglove (topic `uORB` + transform + derivati).

8.3 Modello 3D e scena

Il drone è rappresentato da un modello **URDF** puro (`f550.urdf`), stilizzato ma fedele alle proporzioni dell’F550 (frame, mast GPS, sei bracci con motori ed eliche), con i due bracci anteriori colorati per indicare il fronte. La scelta dell’URDF rispetto a una mesh fotorealistica privilegia riproducibilità, manutenibilità e leggibilità tecnica. È disponibile anche un **overlay satellitare** georeferenziato sul punto di decollo (tile ESRI World Imagery), embeddato direttamente nel `.mcap`.

8.4 Layout dei pannelli

Il layout committato (`foxglove/drone f550.json`) organizza 9 pannelli in due colonne: a sinistra la scena 3D con RPM motori, comandi mixer e input del pilota; a destra i messaggi di log, le transizioni di stato di volo, la qualità del fix GPS e l’altitudine. Durante lo scrubbing si vede simultaneamente il drone muoversi nello spazio e l’evoluzione di tutte le grandezze diagnostiche — la modalità con cui è stata ricostruita la dinamica dell’incidente del Capitolo 6.

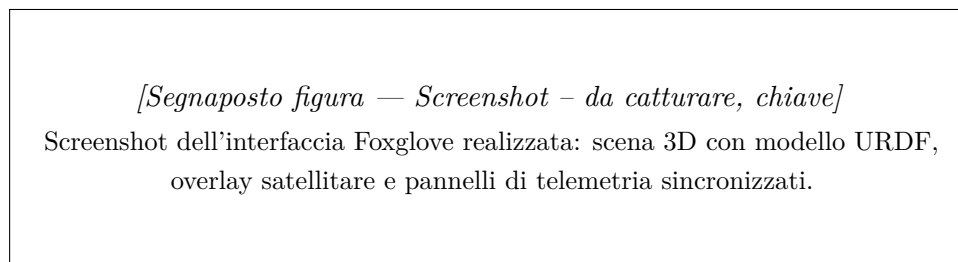


Figura 8.2: Screenshot dell’interfaccia Foxglove realizzata: scena 3D con modello URDF, overlay satellitare e pannelli di telemetria sincronizzati.

Capitolo 9

Conclusioni

Il progetto ha percorso l'intero arco previsto: *mettere in funzione* il drone, *configurarlo* con QGroundControl e PX4, ed *eseguire i test acquisendo dati*. Al termine, l'esacottero F550 è un velivolo volabile e pienamente strumentato, con una catena di acquisizione (147 topic uORB, FIFO inerziali a ~ 1 kHz, telemetria ESC e batteria) adeguata alla diagnosi dello stato di salute.

9.1 Risultati principali

- **Sistema operativo e strumentato:** configurazione completa di sensori, GPS UART, power module, telemetria ESC, failsafe e profilo di logging.
- **Campagna di volo conclusa:** hover baseline e con pala 5 %/10 % (Set A–C), traiettorie quadrate 5 %/10 % (Set D–E) e un blocco extra con payload (Set P).
- **Guasti diagnosticati e risolti:** dalla sostituzione del flight controller alla ricostruzione del cavo GPS (ri-crimpatura e schermatura), che ha eliminato il contatto meccanico intermittente responsabile dei dropout (BER 49 % $\rightarrow$ 0,24 %); resta aperto, come secondo modo di guasto indipendente, lo stallò del driver GPS in modo RTK, per cui la campagna è stata condotta in GPS standard. Ogni caso è stato documentato con la sua diagnosi e azione correttiva.
- **Dati utili:** le metriche estratte (vibrazione, `imbalanced_prop`, rapporto comando/RPM) discriminano il livello di danno alla pala, confermando la bontà dell'acquisizione.
- **Contributo extra:** una pipeline di visualizzazione 3D in Foxglove per il replay dei voli.

9.2 Limiti e sviluppi futuri

La campagna non comprende le traiettorie casuali (Set F/G), non eseguite, né le prove al 15 % (sospese dal docente). Il bias strutturale di montaggio (M3/M6 vs M4/M5) andrebbe caratterizzato e corretto prima di estendere le analisi alle traiettorie. Sul fronte hardware, la ridondanza GPS (secondo modulo M9N) e l'upgrade a un ricevitore multi-banda (ZED-F9P) migliorerebbero robustezza e precisione RTK. Infine, le analisi del Capitolo 7 possono essere

estese con un'analisi FFT sistematica dei FIFO inerziali per legare le firme spettrali al tipo e al livello di danno.

Appendice A

Appendici

Le appendici raccolgono i dettagli fini richiamati dai capitoli ma tenuti fuori dal corpo principale per leggibilità.

A.1 Mappatura dei log per sessione

Le tabelle seguenti mappano i file `.ulg` alle prove del piano voli. Il dettaglio completo (tensioni di batteria, voli di riscaldamento e ripetizioni per batteria scarica) è nei file `log/<data>/README.md` del repository.

Tabella A.1: Set A–C — hover (27/05 e 04/06). Pala a turno su M1...M6.

Prova	Configurazione	Log
A.1–A.6	Pale sane (swap coppie)	2026-05-27/13_56_40 ... 14_15_52
B.1	Pala 5 % su M1	2026-05-27/16_00_57
B.2	Pala 5 % su M2	2026-05-27/15_47_35
B.3	Pala 5 % su M3	2026-05-27/16_03_10
B.4	Pala 5 % su M4	2026-05-27/15_50_20
B.5	Pala 5 % su M5	2026-05-27/15_54_10
B.6	Pala 5 % su M6	2026-05-27/16_05_24
C.1	Pala 10 % su M1	2026-05-27/16_07_29
C.2	Pala 10 % su M2	2026-06-04/12_39_37
C.3	Pala 10 % su M3	2026-06-04/12_21_42
C.4	Pala 10 % su M4	2026-06-04/12_43_47
C.5	Pala 10 % su M5	2026-06-04/12_46_09
C.6	Pala 10 % su M6	2026-06-04/12_28_29

Tabella A.2: Set D ed E — quadrato 8×8 m. Il baseline pale sane è condiviso (13_21_21).

Prova	Log (Set D, 5 %)	Prova	Log (Set E, 10 %)
D.1 (sane)	2026-06-05/13_21_21	E.1/E.2	riusa D.1
D.3/D.4 (M1)	13_25_30 / 13_32_25	E.3/E.4 (M1)	13_27_52 / 13_33_16
D.5/D.6 (M2)	14_08_33 / 14_11_02	E.5/E.6 (M2)	13_52_09 / 13_59_52
D.7/D.8 (M3)	13_37_21 / 13_40_36	E.7/E.8 (M3)	13_37_29 / 13_40_32
D.9/D.10 (M4)	14_17_24 / 14_34_05	E.9/E.10 (M4)	14_15_24 / 14_18_32
D.11 (M5)	2026-06-12/13_20_00	E.11/E.12 (M5)	14_06_38 / 14_09_45
D.13/D.14 (M6)	13_45_18 / 14_05_24	E.13/E.14 (M6)	13_44_15 / 13_47_50

Tabella A.3: Set P — payload (12/06, blocco extra fuori piano A–G).

Prova	Configurazione	Traiettoria	Log
P.1	Payload, pale sane	libera	2026-06-12/14_44_38
P.2	Payload, pale sane	libera	2026-06-12/14_50_12
P.3	Payload, pala 10 % M2	quadrato	2026-06-12/15_29_08
P.4	Payload, pala 10 % M2	quadrato	2026-06-12/15_31_52
P.5	Payload, pala 5 % M2	quadrato	2026-06-12/15_35_21

A.2 Parametri PX4 rilevanti

Tabella A.4: Parametri PX4 principali configurati durante il progetto.

Parametro	Valore	Funzione
UAVCAN_ENABLE	0	Disabilita DroneCAN (GPS su UART)
GPS_1_CONFIG	202	GPS su porta GPS 2
GPS_DUMP_COMM	1	Dump UART grezzo per diagnosi (poi disattivabile)
BAT1_V_CHANNEL	16	Canale ADC tensione batteria
BAT1_I_CHANNEL	17	Canale ADC corrente batteria
DSHOT_CONFIG	600	DSHOT600 verso gli ESC
DSHOT_TEL_CFG	102	Telemetria ESC su TELEM2
SDLOG_PROFILE	849	Profilo di logging (default+HF+3IMU+FIFO)
IMU_GYRO_RATEMAX	1000	Frequenza massima giroscopio (Hz)
EKF2_NOAID_TOUT	10 s	Timeout EKF perdita posizione
COM_POSCTL_NAVL	Altitude	Failsafe perdita posizione in POSCTL
MPC_XY_VEL_MAX	5 m/s	Velocità orizzontale massima
MPC_TILTMAX_AIR	25°	Tilt massimo in volo

A.3 Riferimenti al repository

I documenti di dettaglio sono mantenuti nel repository di progetto: `diario.md` (cronologia), `piano-voli.md` (piano completo), `docs/BOM.md` (BOM), `maintenance/` (troubleshooting e checklist), `analisi/` (analisi pale), `plot/` (script e grafici), `foxglove/` (pipeline di visualizzazione).